



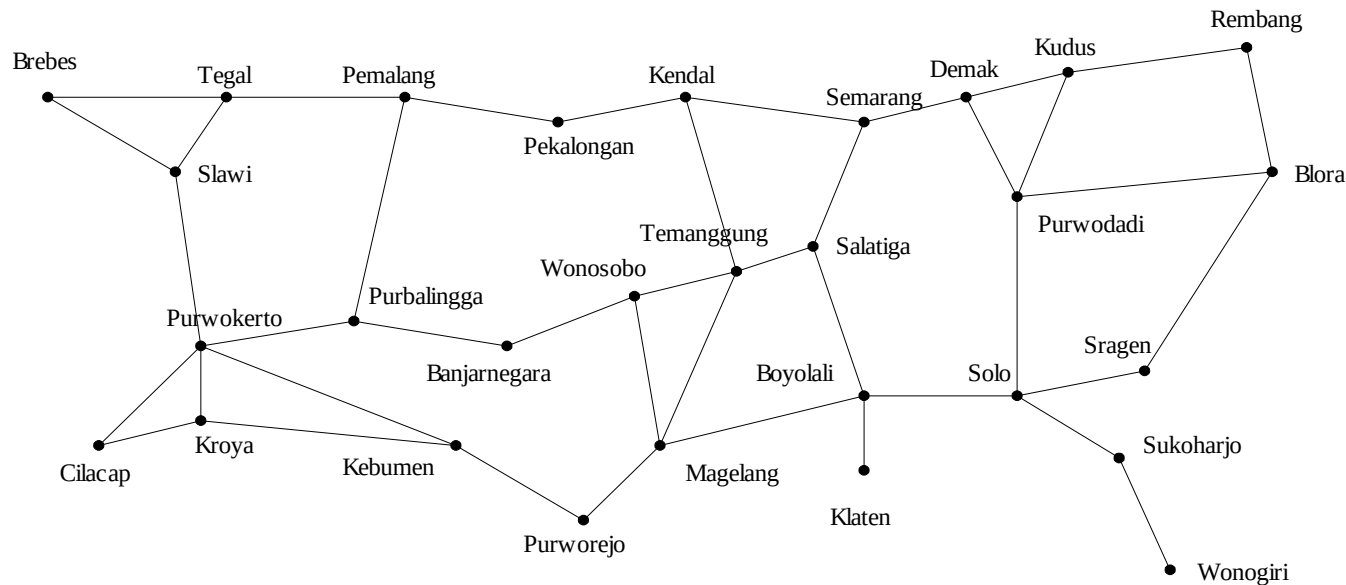
Graf



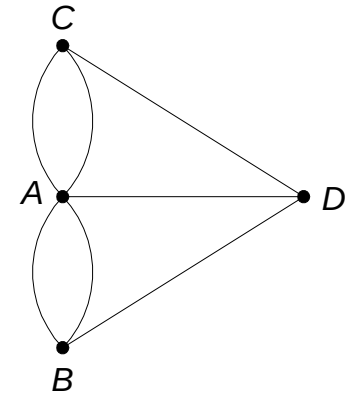
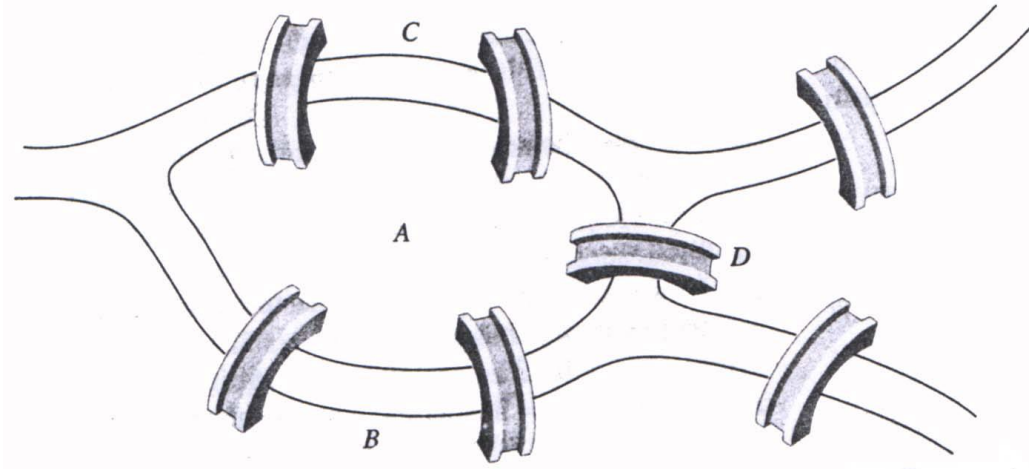
Algoritma & Struktur Data

Pendahuluan

- Graf digunakan untuk merepresentasikan objek-objek diskrit dan hubungan antara objek-objek tersebut.
- Gambar di bawah ini sebuah graf yang menyatakan peta jaringan jalan raya yang menghubungkan sejumlah kota di Provinsi Jawa Tengah.

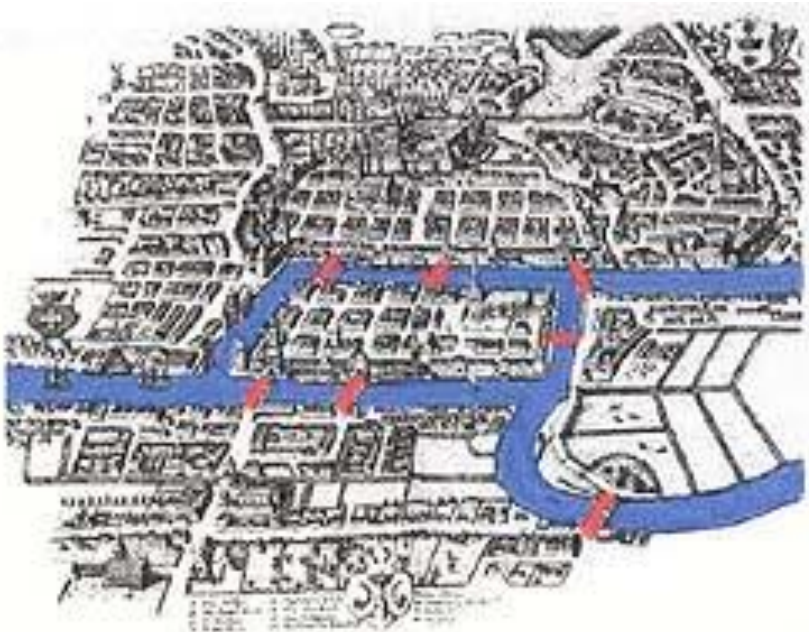


- Sejarah Graf: masalah jembatan Königsberg (tahun 1736)



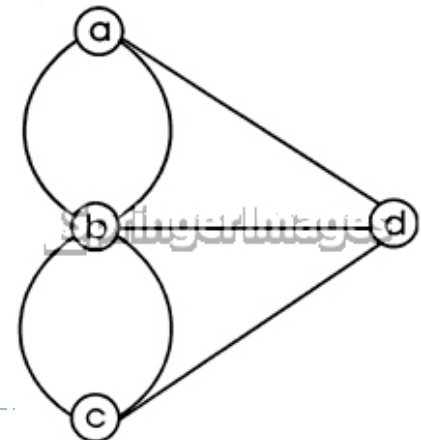
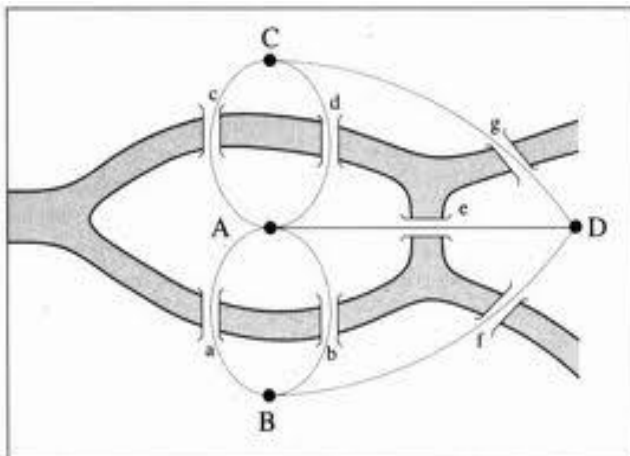
Gambar 1. Masalah Jembatan Königsberg

- Graf yang merepresentasikan jembatan Königsberg:
Simpul (*vertex*) → menyatakan daratan
Sisi (*edge*) → menyatakan jembatan
- Bisakah melalui setiap jembatan tepat sekali dan kembali lagi ke tempat semula?



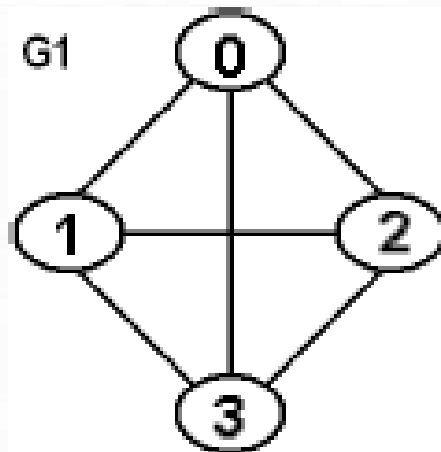
Königsberg Bridge Problem

Leonhard Euler
15 April 1707 – 18 September 1783



Definisi

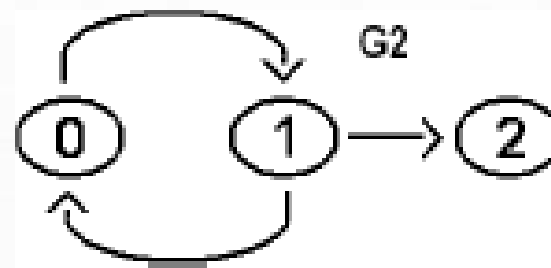
Sebuah **graph** G dibentuk oleh dua himpunan: himpunan verteks dan himpunan edge. Biasanya ditulis dengan notasi $G = (V, E)$



Undirected graph

$$V(G_1) = \{0, 1, 2, 3\}$$

$$E(G_1) = \{ (0, 1), (0, 2), (0, 3), (1, 2), (1, 3), (2, 3) \}$$



Directed graph (digraph)

$$V(G_2) = \{0, 1, 2\}$$

$$E(G_2) = \{ \langle 0, 1 \rangle, \langle 1, 0 \rangle, \langle 1, 2 \rangle \}$$

Terminologi

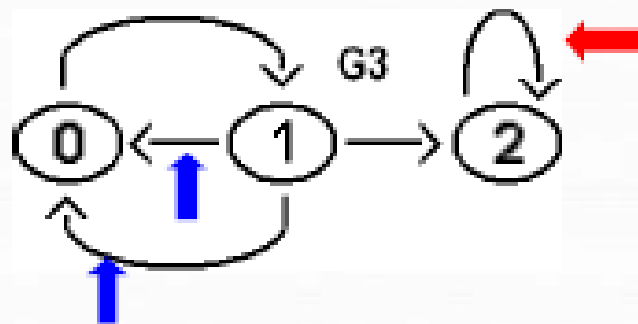
Undirected edge: edge tidak berarah (1, 3) sama dengan (3, 1)

Directed edge: edge berarah $\langle 1, 2 \rangle$ tidak sama dengan $\langle 2, 1 \rangle$

Undirected graph: graph yang semua edge-nya bersifat undirected.

Directed graph (digraph): graph yang semua edge-nya bersifat directed

Simple graph: graph yang tidak mengandung **self loop** dan **multiple edge**



Min

Terminologi

- ▶ **Complete Graph** : graph yang jumlah edge-nya maksimum
 - ▶ Undirected graph : $n(n-1)/2$
 - ▶ Directed graph : $n(n-1)$
- ▶ **Simple path**
 - ▶ Path yang verteks pertama dan terakhir berbeda
- ▶ **Cycle path**
 - ▶ Simple path yang verteks pertama dan terakhir sama
- ▶ **Weighted graph**
 - ▶ Graph yang setiap edgenya diberi bobot (nilai)



Terminologi

- ▶ **Degree (v)**

- ▶ Banyaknya edge yang berhubungan dengan verteks v

- ▶ **Indegree (v)**

- ▶ Banyaknya edge yang masuk (menuju) verteks v pada sebuah graph

- ▶ **Outdegree (V)**

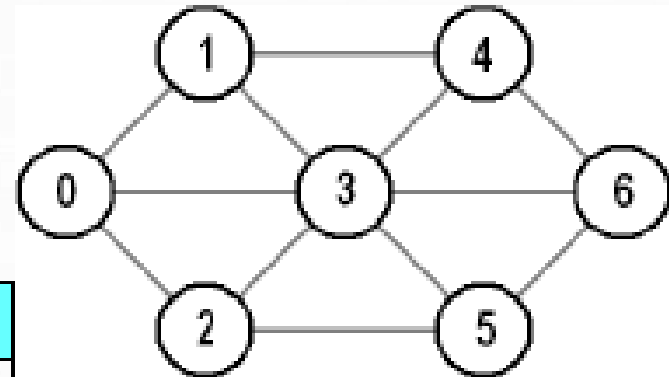
- ▶ Banyaknya edge yang keluar dari verteks v pada sebuah graph



Representasi

Adjacency Matrix
unweighted, undirected

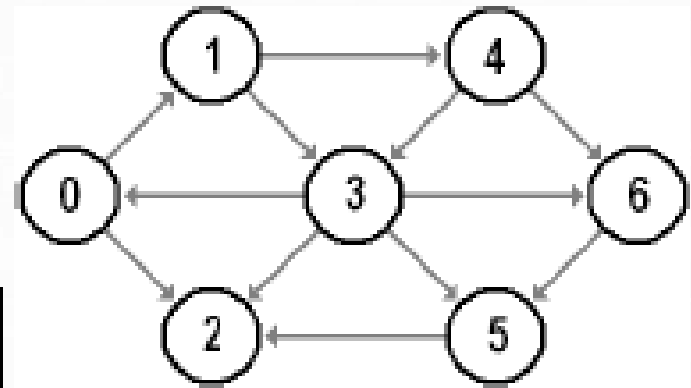
	[0]	[1]	[2]	[3]	[4]	[5]	[6]
[0]	0	1	1	1	0	0	0
[1]	1	0	0	1	1	0	0
[2]	1	0	0	1	0	1	0
[3]	1	1	1	0	1	1	1
[4]	0	1	0	1	0	0	1
[5]	0	0	1	1	0	0	1
[6]	0	0	0	1	1	1	0



Representasi

Adjacency Matrix
unweighted, directed

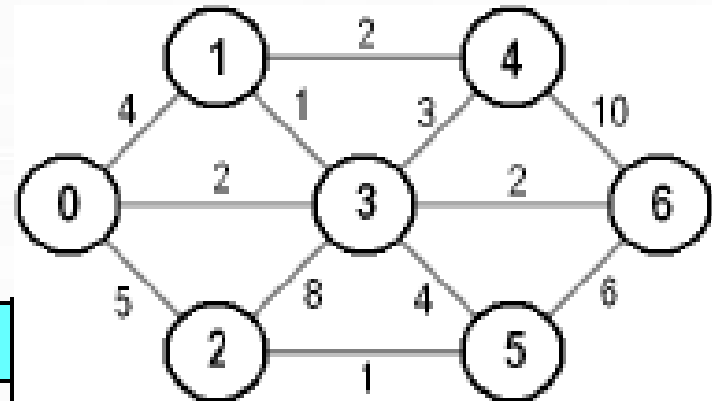
	[0]	[1]	[2]	[3]	[4]	[5]	[6]
[0]	0	1	1	0	0	0	0
[1]	0	0	0	1	1	0	0
[2]	0	0	0	0	0	0	0
[3]	1	0	1	0	0	1	1
[4]	0	0	0	1	0	0	1
[5]	0	0	1	0	0	0	0
[6]	0	0	0	0	0	1	0



Representasi

Adjacency Matrix
weighted, undirected

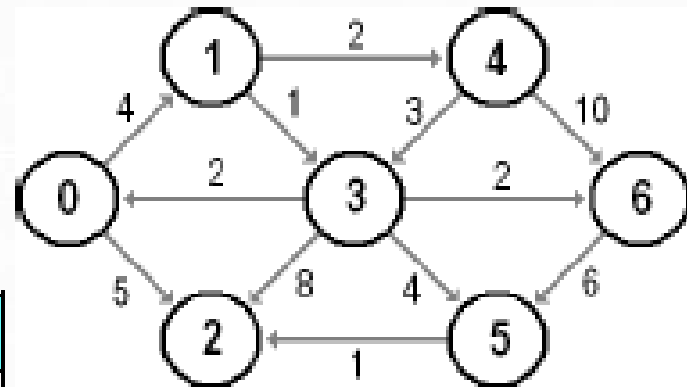
	[0]	[1]	[2]	[3]	[4]	[5]	[6]
[0]	0	4	5	2	0	0	0
[1]	4	0	0	1	2	0	0
[2]	5	0	0	8	0	1	0
[3]	2	1	8	0	3	4	2
[4]	0	2	0	3	0	0	10
[5]	0	0	1	4	0	0	6
[6]	0	0	0	2	10	6	0



Representasi

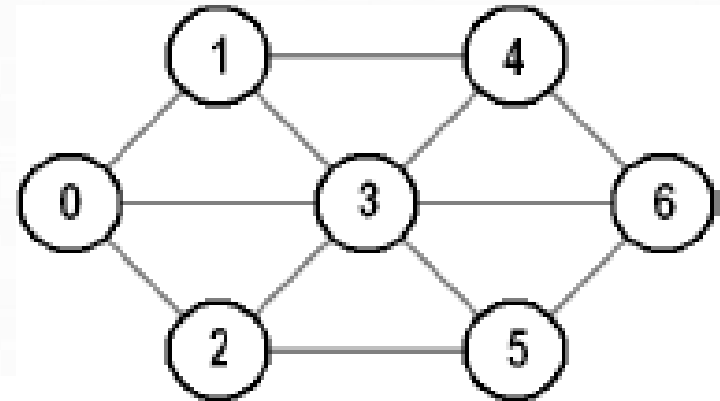
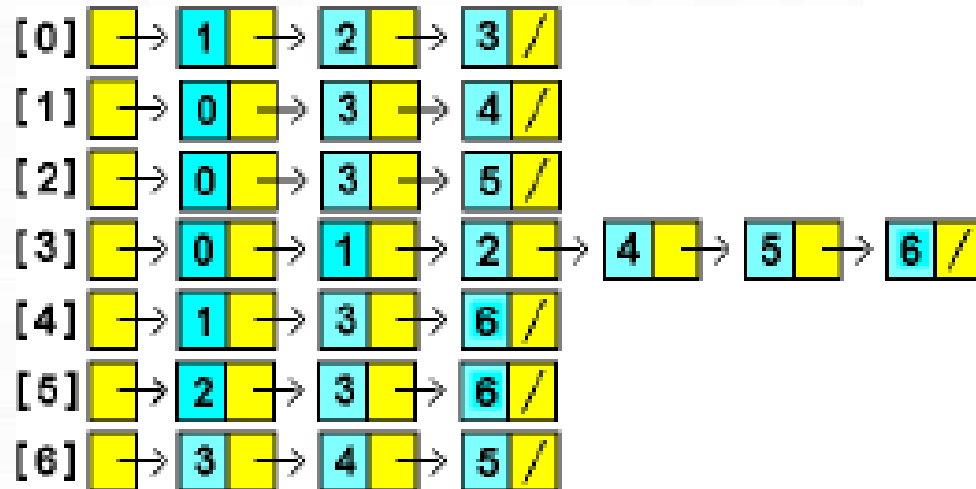
Adjacency Matrix
weighted, directed

	[0]	[1]	[2]	[3]	[4]	[5]	[6]
[0]	Max	4	5	Max	Max	Max	Max
[1]	Max	Max	Max	1	2	Max	Max
[2]	Max	Max	Max	Max	Max	Max	Max
[3]	2	Max	8	Max	Max	4	2
[4]	Max	Max	Max	3	Max	Max	10
[5]	Max	Max	1	Max	Max	Max	Max
[6]	Max	Max	Max	Max	Max	6	Max



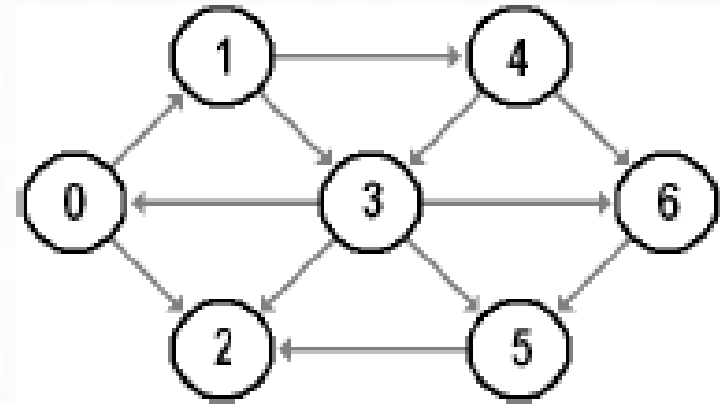
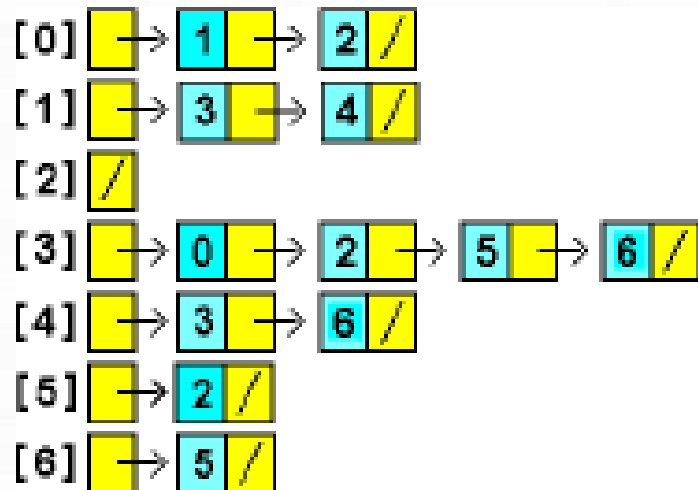
Representasi

Adjacency Lists unweighted, undirected



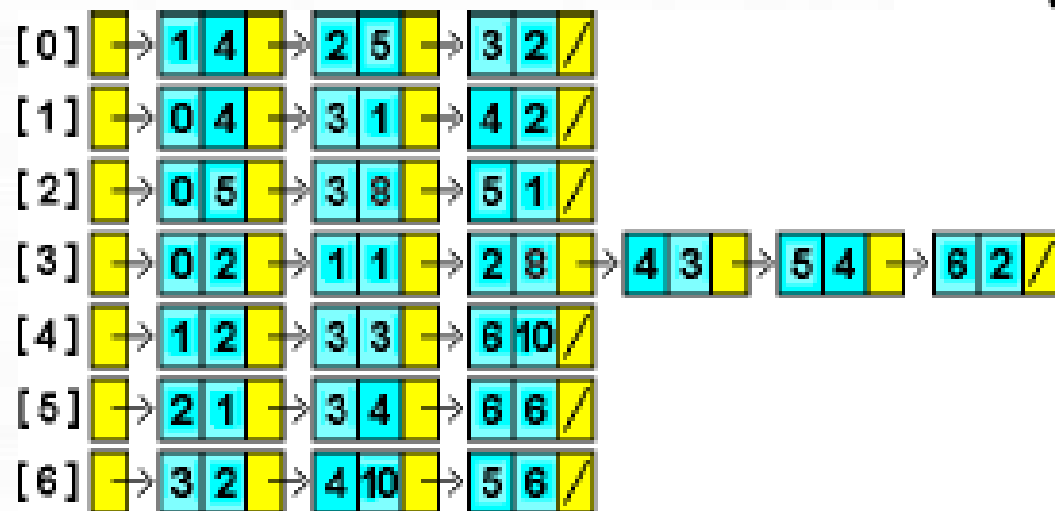
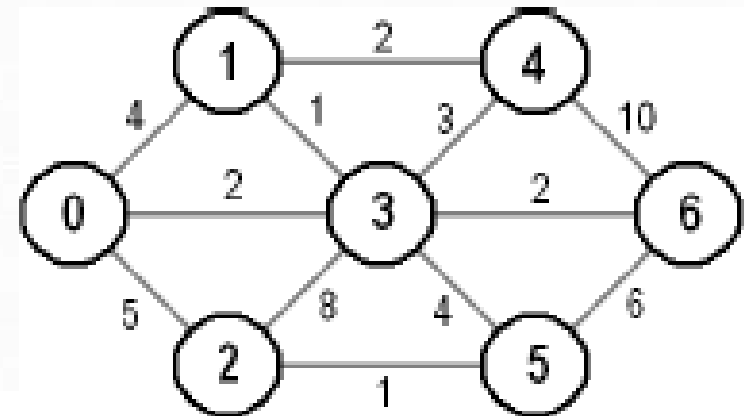
Representasi

Adjacency Lists unweighted, directed



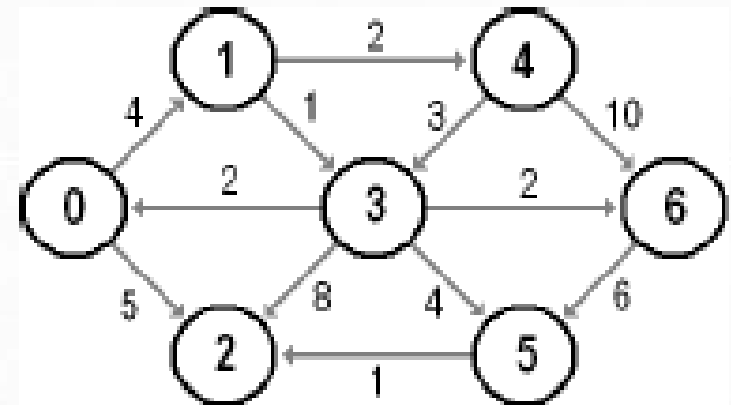
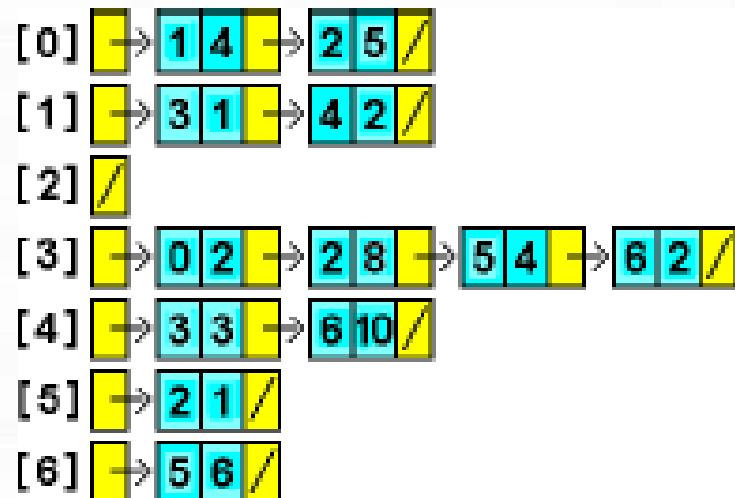
Representasi

Adjacency Lists weighted, undirected



Representasi

Adjacency Lists weighted, directed





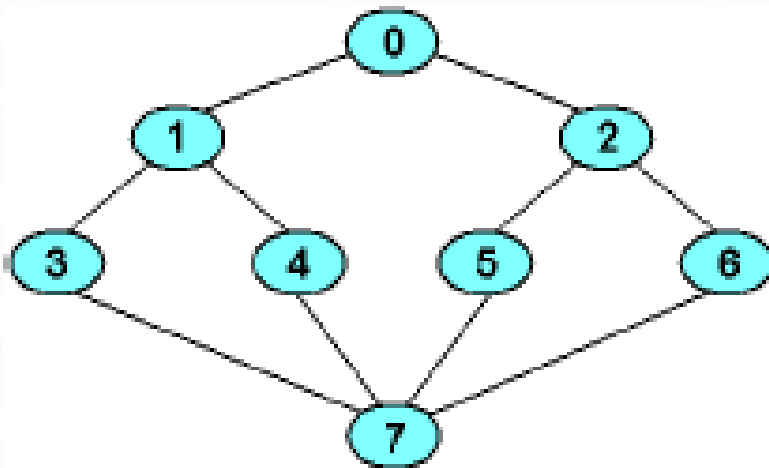
Traversal pada Graph

Breadth-First Search

Breadth-First Search

- ↪ Mengunjungi seluruh verteks satu persatu
- ↪ Ke samping (selevel) baru ke dalam (level berikutnya)
- ↪ Disebut juga breadth-first traversal
- ↪ Dengan bantuan queue

Breadth-First Search

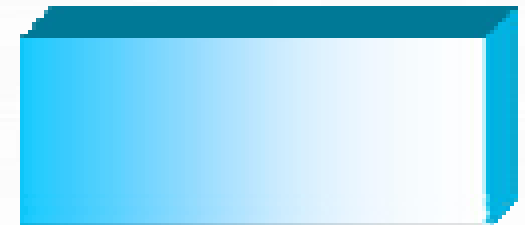


[0] → 1 → 2 /
[1] → 0 → 3 → 4 /
[2] → 0 → 5 → 6 /
[3] → 1 → 7 /
[4] → 1 → 7 /
[5] → 2 → 7 /
[6] → 2 → 7 /
[7] → 3 → 4 → 5 → 6 /

Breadth-First Search

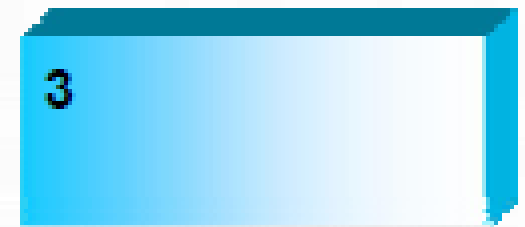
Visited [0] [1] [2] [3] [4] [5] [6] [7]

0	0	0	0	0	0	0	0
---	---	---	---	---	---	---	---



[0] [1] [2] [3] [4] [5] [6] [7]

0	0	0	1	0	0	0	0
---	---	---	---	---	---	---	---



3

[3] → 1 → 7 /

1 7



Breadth-First Search

Visited [0] [1] [2] [3] [4] [5] [6] [7]

0	0	0	1	0	0	0	0
---	---	---	---	---	---	---	---

3

[0] [1] [2] [3] [4] [5] [6] [7]

0	1	0	1	0	0	0	0
---	---	---	---	---	---	---	---

3 1

1 7

7

[1] → 0 → 3 → 4 /

7 0 4

Breadth-First Search

Visited [0] [1] [2] [3] [4] [5] [6] [7]

0	1	0	1	0	0	0	0
---	---	---	---	---	---	---	---

3 1

[0] [1] [2] [3] [4] [5] [6] [7]

0	1	0	1	0	0	0	1
---	---	---	---	---	---	---	---

3 1 7

7 0 4

0 4

[7] → 3 → 4 → 5 → 6 /

0 4 4 5 6

Breadth-First Search

Visited [0] [1] [2] [3] [4] [5] [6] [7]

0	1	0	1	0	0	0	1
---	---	---	---	---	---	---	---

3 1 7

[0] [1] [2] [3] [4] [5] [6] [7]

1	1	0	1	0	0	0	1
---	---	---	---	---	---	---	---

3 1 7 0

0	4	4	5	6
---	---	---	---	---

4	4	5	6
---	---	---	---

[0] → 1 → 2 /

4	4	5	6	2
---	---	---	---	---

Breadth-First Search

visited [0] [1] [2] [3] [4] [5] [6] [7]

1	1	0	1	0	0	0	1
---	---	---	---	---	---	---	---

3 1 7 0

[0] [1] [2] [3] [4] [5] [6] [7]

1	1	0	1	1	0	0	1
---	---	---	---	---	---	---	---

3 1 7 0 4

4 4 5 6 2

4 5 6 2

[4] → 1 → 7 /

4 5 6 2

Breadth-First Search

Visited	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]
	1	1	0	1	1	0	0	1

4 5 6 2

5 6 2

[4] → 1 → 7 /

5 6 2

3 1 7 0 4

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]
1	1	0	1	1	0	0	1

3 1 7 0 4

Breadth-First Search

visited [0] [1] [2] [3] [4] [5] [6] [7]

1	1	0	1	1	0	0	1
---	---	---	---	---	---	---	---

3 1 7 0 4

[0] [1] [2] [3] [4] [5] [6] [7]

1	1	0	1	1	1	0	1
---	---	---	---	---	---	---	---

3 1 7 0 4 5

5 6 2

6 2

[5] → 2 → 7 /

6 2 2

Breadth-First Search

Visited [0] [1] [2] [3] [4] [5] [6] [7]

1	1	0	1	1	1	0	1
---	---	---	---	---	---	---	---

3 1 7 0 4 5

[0] [1] [2] [3] [4] [5] [6] [7]

1	1	0	1	1	1	1	1
---	---	---	---	---	---	---	---

3 1 7 0 4 5 6

6 2 2

2 2

[6] → 2 → 7 /

2 2 2



Breadth-First Search

Visited [0] [1] [2] [3] [4] [5] [6] [7]

1	1	0	1	1	1	1	1
---	---	---	---	---	---	---	---

3 1 7 0 4 5 6

[0] [1] [2] [3] [4] [5] [6] [7]

1	1	1	1	1	1	1	1
---	---	---	---	---	---	---	---

3 1 7 0 4 5 6 2

2 2 2

2 2

[2] → 0 → 5 → 6 /

2 2

Breadth-First Search

Visited [0] [1] [2] [3] [4] [5] [6] [7]

1	1	1	1	1	1	1	1
---	---	---	---	---	---	---	---

3 1 7 0 4 5 6 2

[0] [1] [2] [3] [4] [5] [6] [7]

1	1	1	1	1	1	1	1
---	---	---	---	---	---	---	---

3 1 7 0 4 5 6 2

2 2

2

[2] → 0 → 5 → 6 /

2



Breadth-First Search

Visited [0] [1] [2] [3] [4] [5] [6] [7]

1	1	1	1	1	1	1	1
---	---	---	---	---	---	---	---

3 1 7 0 4 5 6 2

[0] [1] [2] [3] [4] [5] [6] [7]

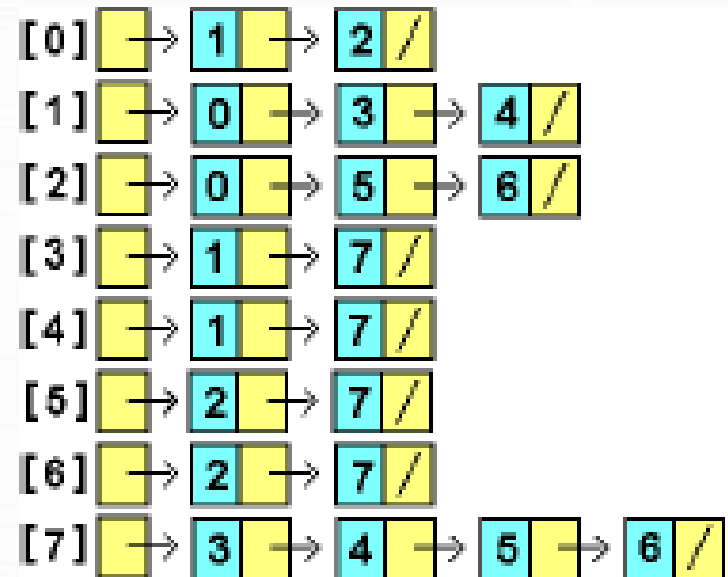
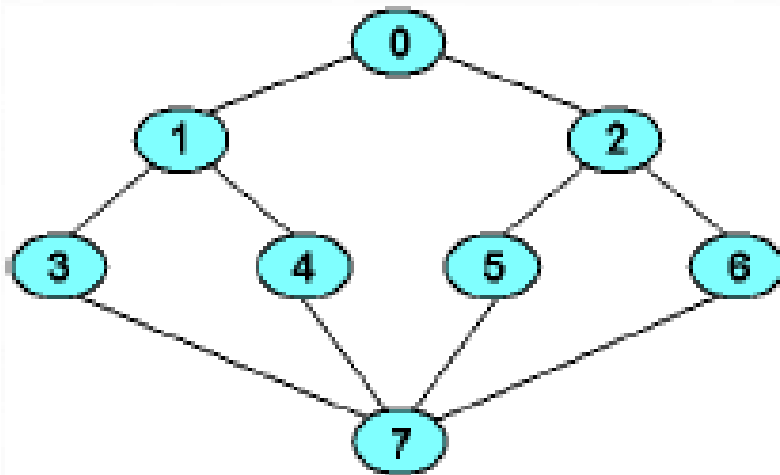
1	1	1	1	1	1	1	1
---	---	---	---	---	---	---	---

3 1 7 0 4 5 6 2

[2] → 0 → 5 → 6 /

SELESAI

Breadth-First Search



Mulai dari verteks 3: 3 1 7 0 4 5 6 2



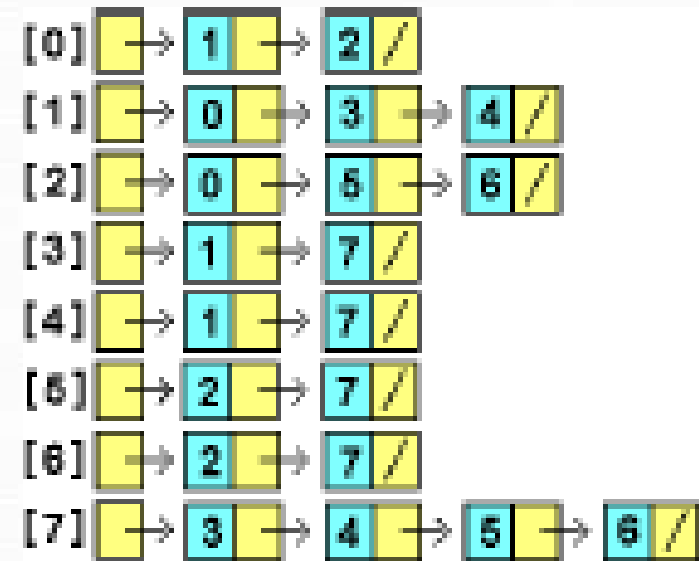
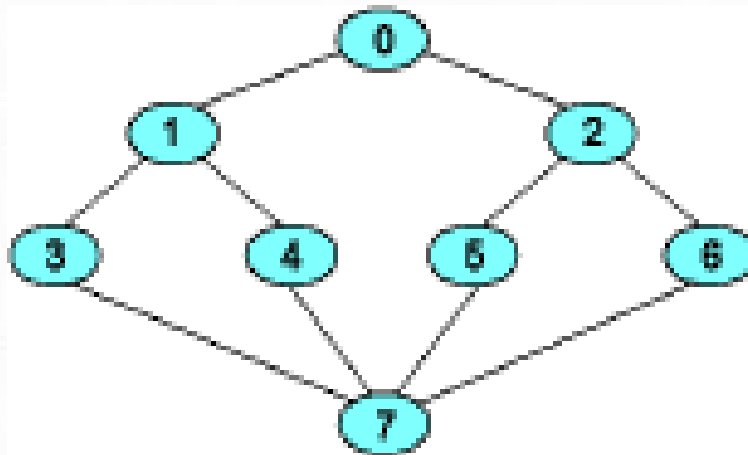
Traversal pada Graph

Depth-First Search

Depth-First Search

- ☞ Mengunjungi seluruh verteks satu persatu
- ☞ Ke dalam baru ke samping
- ☞ Disebut juga depth-first traversal
- ☞ Dengan cara recursive

Depth-First Search



Depth-First Search

[0] → 1 → 2 /



visited [0] [1] [2] [3] [4] [5] [6] [7]

1	0	0	0	0	0	0	0
---	---	---	---	---	---	---	---

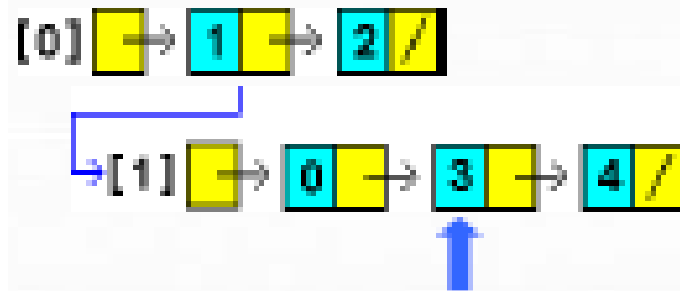
0

DFS

visited [0] [1] [2] [3] [4] [5] [6] [7]

1	1	0	0	0	0	0	0
---	---	---	---	---	---	---	---

0 1

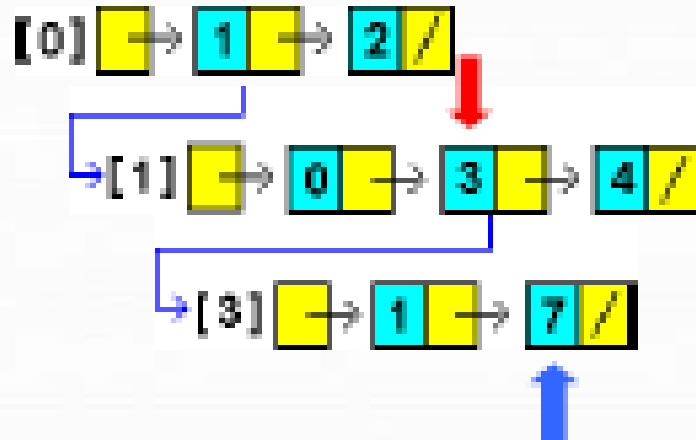


DFS

visited [0] [1] [2] [3] [4] [5] [6] [7]

1	1	0	1	0	0	0	0
---	---	---	---	---	---	---	---

0 1 3

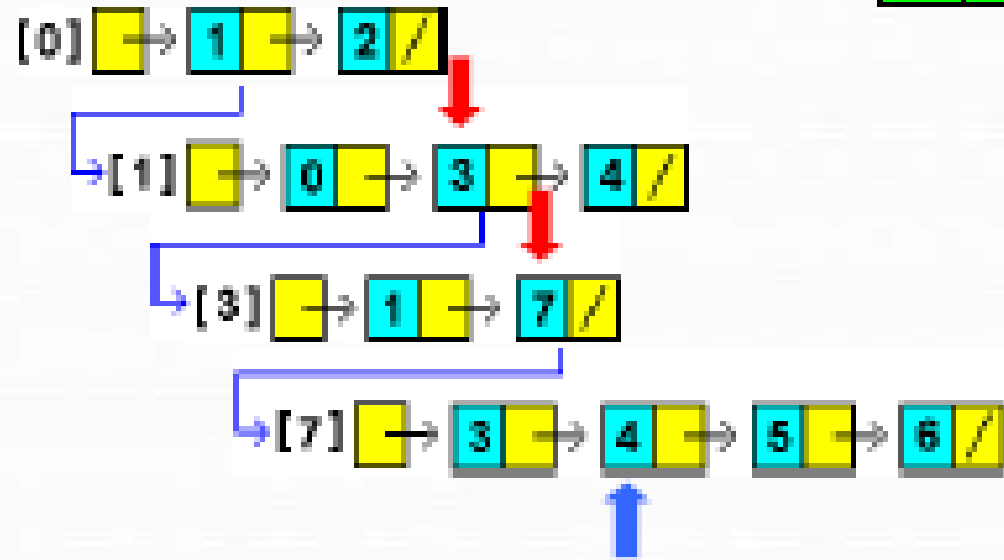


DFS

visited [0] [1] [2] [3] [4] [5] [6] [7]

1	1	0	1	0	0	0	1
---	---	---	---	---	---	---	---

0 1 3 7

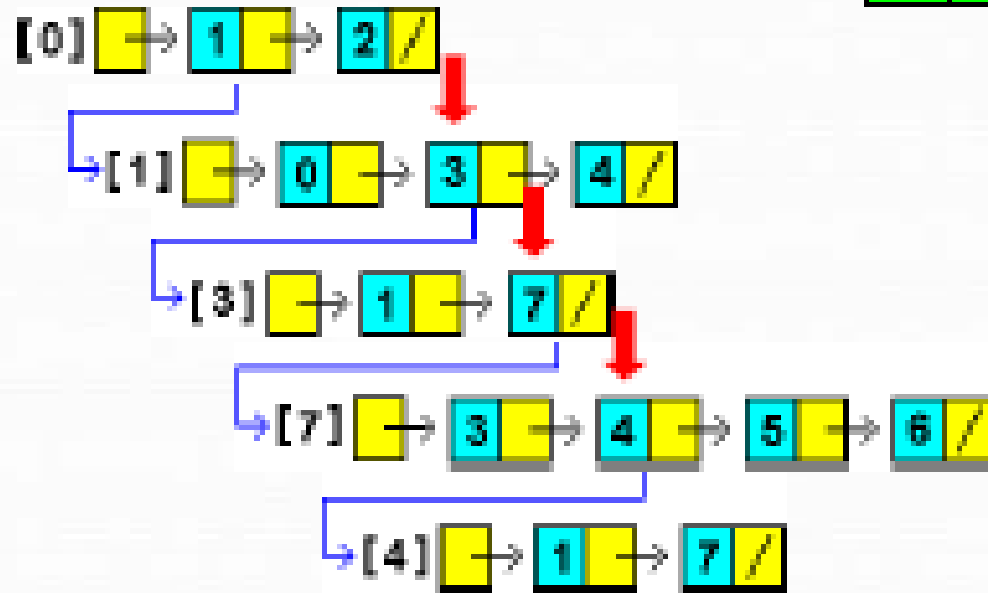


DFS

visited [0] [1] [2] [3] [4] [5] [6] [7]

1	1	0	1	1	0	0	1
---	---	---	---	---	---	---	---

0 1 3 7 4

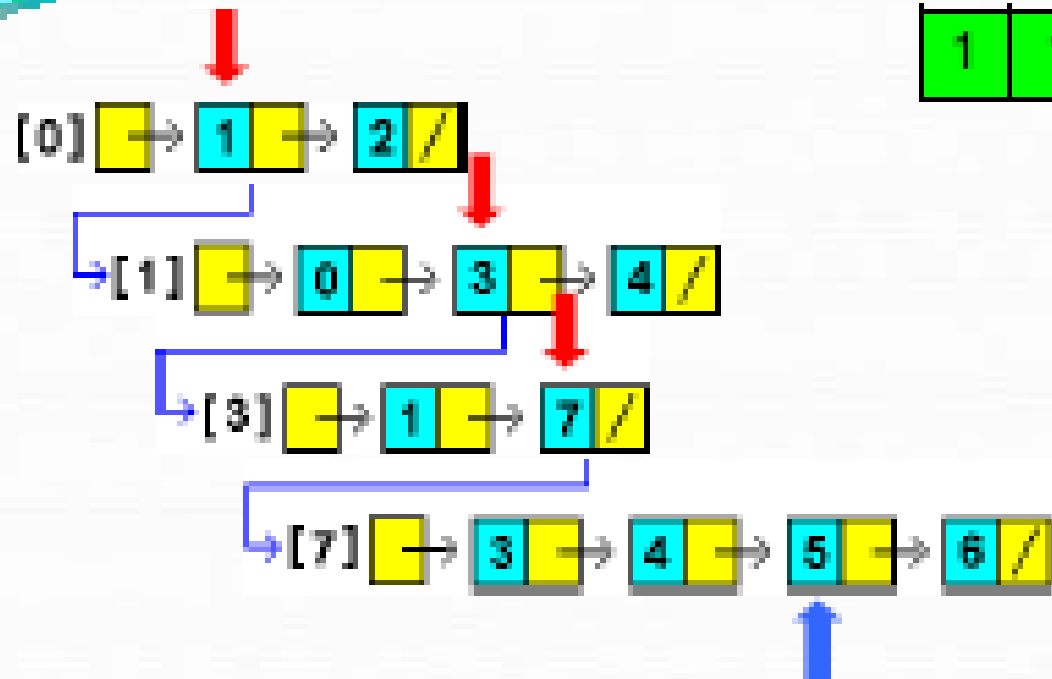


DFS

visited [0] [1] [2] [3] [4] [5] [6] [7]

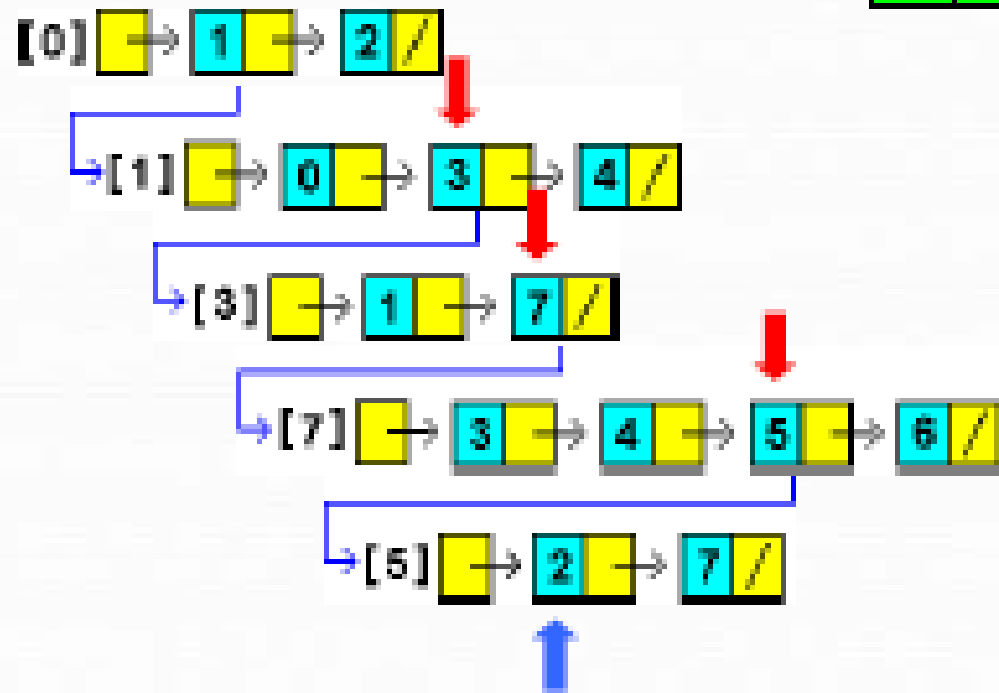
1	1	0	1	1	0	0	1
---	---	---	---	---	---	---	---

0 1 3 7 4



1	1	0	1	1	1	0	1
---	---	---	---	---	---	---	---

0 1 3 7 4 5

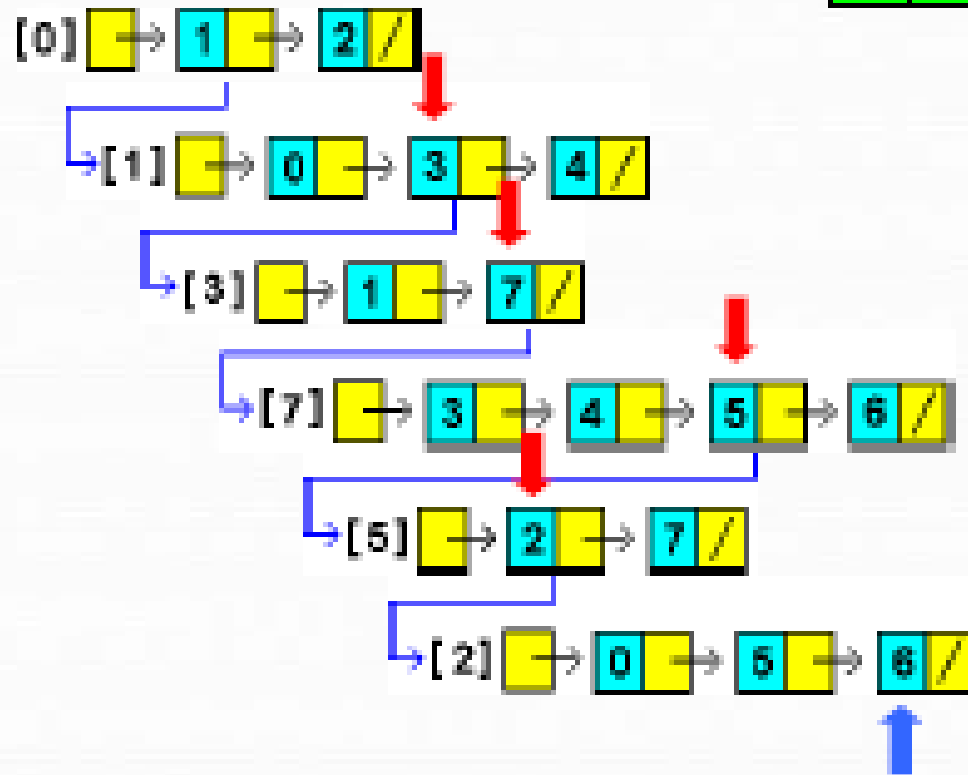


DFS

visited [0] [1] [2] [3] [4] [5] [6] [7]

1	1	1	1	1	1	0	1
---	---	---	---	---	---	---	---

0 1 3 7 4 5 2

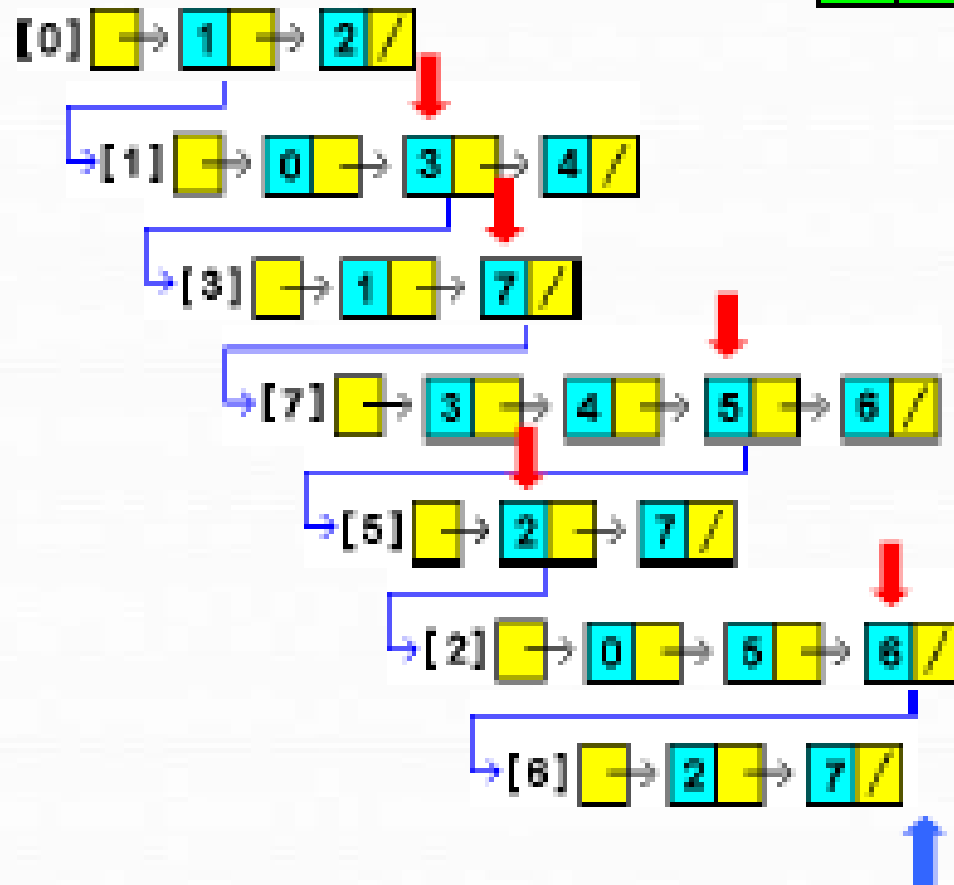


DFS

visited [0] [1] [2] [3] [4] [5] [6] [7]

1	1	1	1	1	1	1	1
---	---	---	---	---	---	---	---

0 1 3 7 4 5 2 6

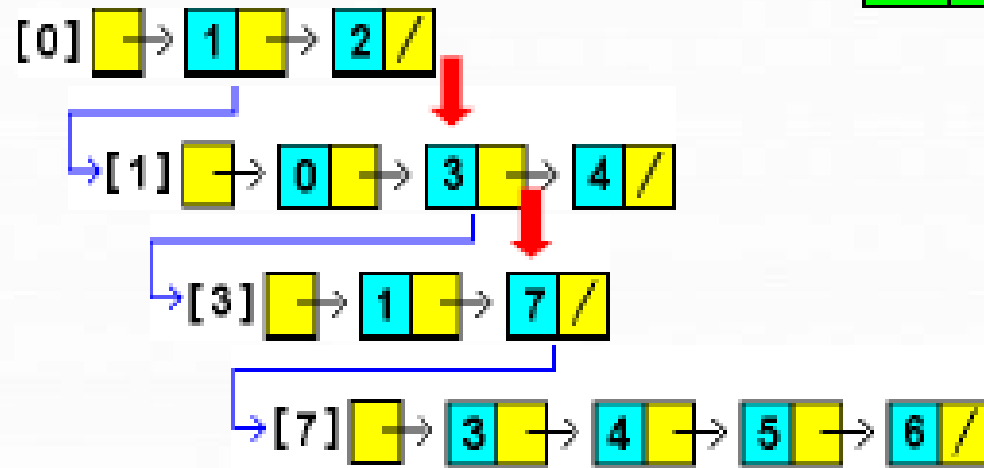


DFS

visited [0] [1] [2] [3] [4] [5] [6] [7]

1	1	1	1	1	1	1	1
---	---	---	---	---	---	---	---

0 1 3 7 4 5 2 6

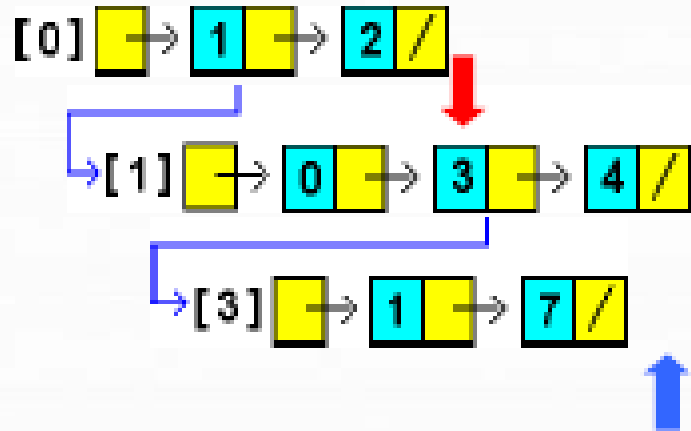


DFS

visited [0] [1] [2] [3] [4] [5] [6] [7]

1	1	1	1	1	1	1	1
---	---	---	---	---	---	---	---

0 1 3 7 4 5 2 6



DFS

visited

[0] [1] [2] [3] [4] [5] [6] [7]

1	1	1	1	1	1	1	1
---	---	---	---	---	---	---	---

0 1 3 7 4 5 2 6

[0] → 1 → 2 /

→ [1] → 0 → 3 → 4 /



DFS

visited [0] [1] [2] [3] [4] [5] [6] [7]

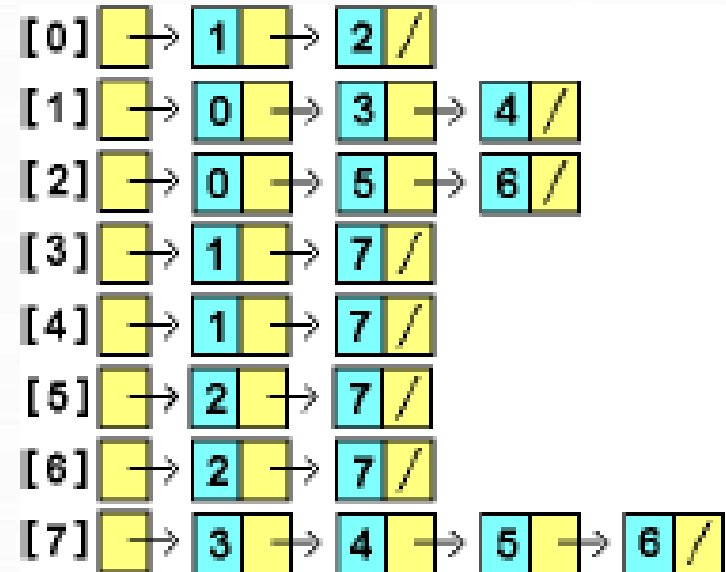
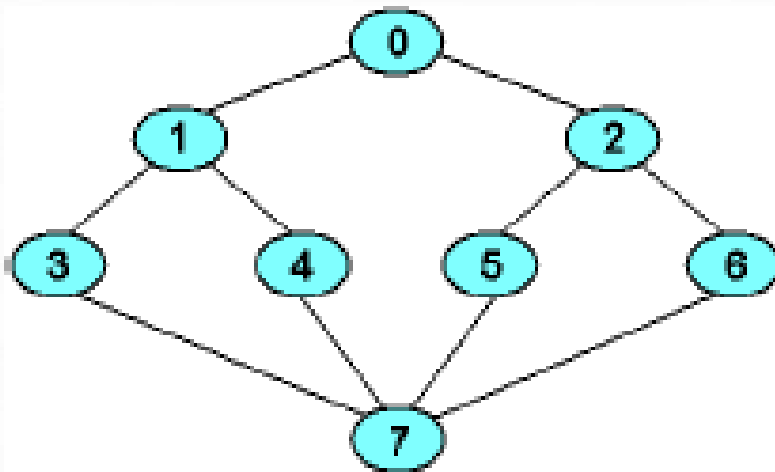
1	1	1	1	1	1	1	1
---	---	---	---	---	---	---	---

[0] → 1 → 2 /



0 1 3 7 4 5 2 6

DFS



Mulai dari verteks 0: 0 1 3 7 4 5 2 6

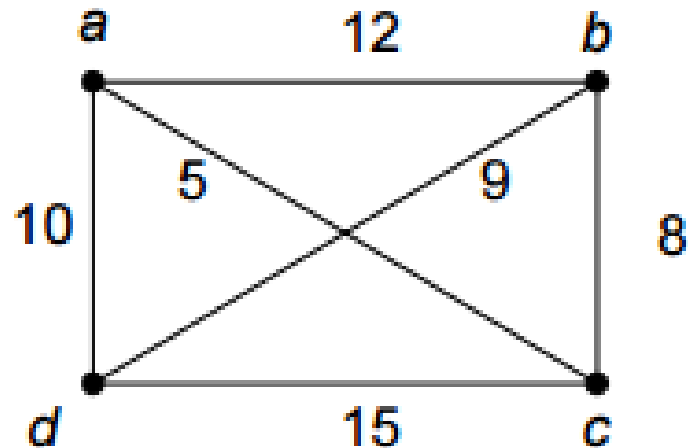
Traveling Salesman Problem

Travelling Salesman Problem (TSP)

- ▶ Permasalahan TSP adalah seorang *salesman* harus mengunjungi semua kota dimana tiap kota hanya dikunjungi sekali, dan dia harus mulai dan ke kembali ke kota asal
- ▶ Tujuan:
 - ▶ Menentukan rute dengan jarak total dan biaya paling minimum
- ▶ Beberapa metode yang dapat digunakan untuk menyelesaikan permasalahan ini adalah:
 - ▶ Complete Enumeration
 - ▶ Greedy Heuristik
 - ▶ Branch and Bound
 - ▶ Dynamic Programming



Complete Enumeration



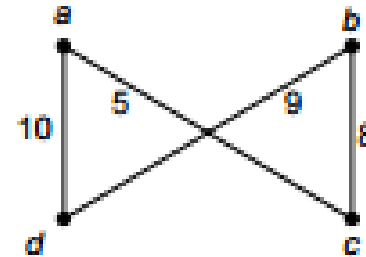
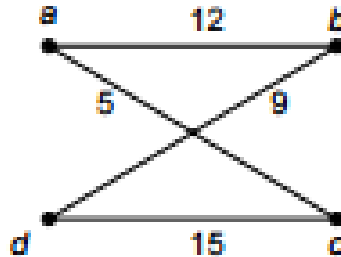
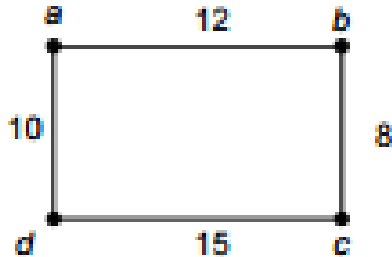
- ▶ Diberikan 4 buah kota (a,b,c,d), serta diketahui jarak antara setiap kota satu sama lain.
- ▶ Temukan rute perjalanan terpendek yang melalui setiap kota hanya sekali dan kembali lagi ke kota asal!

Penyelesaian TSP dengan **Complete Enumeration**

- ▶ Metode ini akan mengenumerasi setiap kemungkinan yang terdapat dalam graf
- ▶ Setelah itu algoritma ini akan membandingkan lintasan mana paling minimum



Penyelesaian TSP dengan **Complete Enumeration** (2)



- ▶ Lintasan 1 (a,b,c,d,a) atau (a,d,c,b,a)
 - ▶ Panjang lintasan = $10 + 12 + 8 + 15 = 45$
- ▶ Lintasan 2 (a,c,d,b,a) atau (a,b,d,c,a)
 - ▶ Panjang lintasan = $12 + 5 + 9 + 15 = 41$
- ▶ Lintasan 3 (a,c,b,d,a) atau (a,d,b,c,a)
 - ▶ Panjang lintasan = $10 + 5 + 9 + 8 = 32$

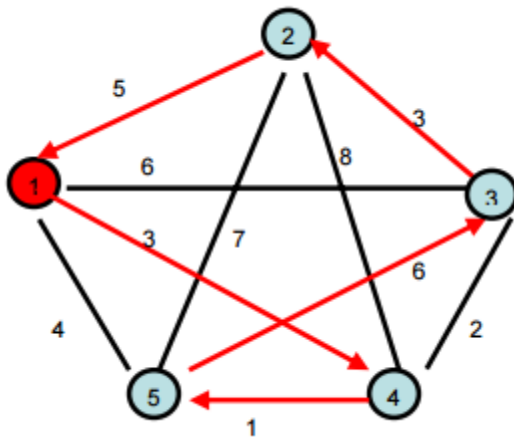


Penyelesaian TSP dengan **Complete Enumeration (3)**

- ▶ Dari hasil enumerasi, didapat hasil minimum, yaitu lintasan 3 (a,c,b,d,a) atau (a,d,b,c,a) dengan panjang lintasan **32**
- ▶ Tetapi jumlah enumerasi dari algoritma ini adalah **$(n-1)!$**
- ▶ Tidak efisien jika jumlah n bernilai sangat besar



Penyelesaian TSP dengan **Greedy Heuristik**



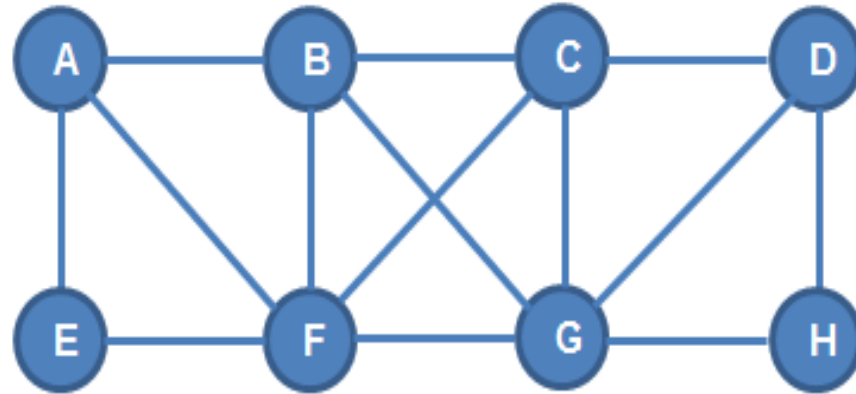
- ▶ Pada Algoritma ini, pemilihan lintasan akan dimulai pada lintasan yang memiliki nilai paling minimum
- ▶ Setiap mencapai suatu kota, algoritma ini akan memilih kota selanjutnya yang belum dikunjungi dan memiliki jarak yang paling minimum
- ▶ Algoritma ini disebut juga *Nearest Neighbour*
- ▶ Kompleksitas algoritma ini memang sangat mengagumkan, yaitu $O(n)$, tetapi hasil yang kita dapat bisa sangat jauh dari hasil yang optimal

Referensi

- ▶ Ngoen, T.S.. 2004. Data Structures. IBI Lecture Notes
- ▶ Mark Allen Weiss. 1997. Data Structures and Algorithm Analysis in C. Addison Wesley Longman, inc.
- ▶ Munir, Rinaldi. 2009. Graf. ITB Lecture Notes
- ▶ Aulia Rahma Amin, Mukhamad Ikhsan, Lastiko Wibisono. Travelling Salesman Problem. ITB



Latihan



- ▶ Untuk graf berikut:
 - ▶ Buatlah adjacency listnya.
 - ▶ Lakukan traversal DFS dan BFS. Node awal adalah A.

